INTEGRANTES:

FELIPE HENRIQUE GONCALVES DE MEDEIROS / RGM:47509716

OSVALDO RAMALHO DE SOUSA JUNIOR / RGM:45949271

JOÃO VITOR VICENTE MENDES/ RGM:45968632

1- PROMPT VAGO (VERSÃO NEUTRA)

Python Executar

```python
# Solicita a quantidade de notas
quantidade = int(input("Quantas notas deseja inserir? "))

soma = 0

# Loop para coletar as notas
for i in range(quantidade):
    nota = float(input(f"Digite a nota {i+1}: "))
    soma += nota

# Cálculo da média
media = soma / quantidade

# Resultado
print(f"\nA média do aluno é: {media:.2f}")

# Situação do aluno (opcional)
if media >= 7:
    print("Situação: Aprovado 🟢")
else:
    print("Situação: Reprovado 🔴")
```

2- VERSÃO PERSONA

def calcular_media(notas): if not notas: raise ValueError("A lista de notas não pode estar vazia.") return sum(notas) / len(notas)

def obter_notas(): while True: try: quantidade = int(input("Informe a quantidade de notas: ")) if quantidade <= 0: print("A quantidade deve ser maior que zero.") continue break except ValueError: print("Entrada inválida. Digite um número inteiro.")

```
notas = []

for i in range(quantidade):
    while True:
        try:
            nota = float(input(f"Digite a nota {i + 1}: "))
            if nota < 0 or nota > 10:
                print("A nota deve estar entre 0 e 10.")
                continue
            notas.append(nota)
            break
        except ValueError:
            print("Entrada inválida. Digite um número válido.")

return notas
```

def classificar_aluno(media): if media >= 7: return "Aprovado 🟢 " elif media >= 5: return "Recuperação 🟡 " else: return "Reprovado 🔴 "

def main(): print("=== Sistema de Cálculo de Média ===")

```
notas = obter_notas()
media = calcular_media(notas)
situacao = classificar_aluno(media)

print("\n--- Resultado ---")
print(f"Notas: {notas}")
print(f"Média: {media:.2f}")
```

```
print(f"Situação: {situacao}")
```

if **name** == "**main**": main()]

3- VERSÃO RESTRITIVA

from **future** import annotations

from dataclasses import dataclass from typing import Iterable, List from statistics import fmean import logging

logging.basicConfig(level=logging.INFO, format="%(levelname)s: %(message)s")

class DomainError(ValueError): pass

@dataclass(frozen=True, slots=True) class Grade: value: float

```
def __post_init__(self) -> None:
    if not isinstance(self.value, (int, float)):
        raise DomainError("Grade must be numeric.")
    if not 0.0 <= float(self.value) <= 10.0:
        raise DomainError("Grade must be between 0 and 10.")
```

class StudentAverageService: APPROVED = "APPROVED" RECOVERY = "RECOVERY" FAILED = "FAILED"

```
@staticmethod
def compute(grades: Iterable[Grade]) -> float:
    values = [g.value for g in grades]
    if not values:
        raise DomainError("Empty grade list.")
    return fmean(values)

@staticmethod
def classify(avg: float) -> str:
    if avg >= 7.0:
        return StudentAverageService.APPROVED
    if avg >= 5.0:
        return StudentAverageService.RECOVERY
```

```
    return StudentAverageService.FAILED
```

class InputAdapter: @staticmethod def read_grades() -> List[Grade]: while True: try: n = int(input()) if n <= 0: raise DomainError break except Exception: continue

```
    grades: List[Grade] = []

    for _ in range(n):
        while True:
            try:
                g = float(input())
                grades.append(Grade(g))
                break
            except Exception:
                continue

    return grades
```

def main() -> None: try: grades = InputAdapter.read_grades() avg = StudentAverageService.compute(grades) status = StudentAverageService.classify(avg) print(f"{avg:.2f}") print(status) except Exception as e: logging.error(str(e))

if **name** == "**main**": main()

ANÁLISE FINAL:

É fascinante observar como a IA reagiu a cada nível de estresse: na versão **Vaga**, ela entregou o básico funcional; na **Persona**, ela

buscou uma estrutura organizada (limpa); e na **Restritiva**, ela "explodiu" a complexidade, usando padrões de arquitetura avançados (como *Dataclasses* e *Services*) para compensar a falta de explicações em texto.

Tabela de Avaliação: O Laboratório de Arquitetura

Critério de Avaliação	Versão Neutra (Vaga)	Versão Persona (Sênior)	Versão Restritiva
Precisão Lógica	**Alta.** O cálculo está correto e o loop funciona bem. Porém, falha se o usuário digitar letras ou números negativos.	**Excelente.** Inclui tratamento de erros (`try/except`), validação de intervalo de notas (0 a 10) e evita divisão por zero.	**Impecável.** Utiliza a biblioteca `statistics` para precisão e valida tipos de dados de forma rígida através de uma classe `Grade`.
Clareza e Eficiência	**Direta.** Código simples e fácil de ler, ideal para iniciantes. Não tem "voltas", mas falta robustez profissional.	**Equilibrada.** Código modularizado em funções (`obter_notas`, `classificar`). É limpo, organizado e fácil de manter.	**Baixa Clareza / Alta Complexidade.** Para um problema simples de média, a IA usou "Overengineering" (excesso de engenharia), criando classes e decoradores desnecessários.
Taxa de Alucinação	**Zero.** Seguiu a instrução básica sem adicionar comportamentos estranhos.	**Baixa.** A "persona" se manifestou na estrutura organizada, mas não houve erros de lógica ou textos desconexos.	**Média.** A IA interpretou "restritiva" como "arquitetura corporativa pesada". Incluiu bibliotecas como `logging` e `dataclasses` que não foram pedidas.